LEAN 4

1. Why Learn Lean 4?

Proving a mathematical claim, for example, the Collatz conjecture (see: https://en.wikipedia.org/wiki/Collatz_conjecture), ensures that it is true, assuming the proof system in question is sound. For the definition of soundness, see:

https://cs.uwaterloo.ca/~eblais/cs245e/f25/pl-soundness

In long proofs, it is hard to verify every step. Verifying with hand-written notes is prone to mistakes. One such example:

https://www.youtube.com/watch?v=MGp2FqXNYaQ

Lean 4 is an interactive proof assistant which allows us to verify a proposed proof. If a proof compiles in Lean 4 and depends only on the standard axioms (more on the standard axioms of Lean 4 later in Section 9), then the statement is definitely true. Or is it?

There is one little caveat. We trust that Lean 4 is correct. The most important trusted component is the Lean 4 kernel, a tiny, well-audited program (More on the Lean 4 kernel in Section 11). The fact that the Lean kernel is small (the total number of lines as well as the file size are small) makes it easier for it to be audited by Lean developers (people who write and test and distribute the source code of Lean). There have been bugs found in Lean 4:

- The Lean4Lean project found a bug:

  https://arxiv.org/abs/2403.14064.

- An AI-assisted "disproof" of the Collatz conjecture:

  https://infosec.exchange/@0xabad1dea/117002106099986943

  https://github.com/leanprover/lean4/issues/14576

  https://leodemoura.github.io/blog/2026-8-1-postmortem-for-kernel-soundness-bug-14576/

- More bugs, fixed in Lean Version 4.33.1:

  https://leodemoura.github.io/blog/2026-8-24-postmortem-for-the-kernel-soundness-bug-hunt/.

Having a bug in a system implies that some developer of Lean made a mistake while writing its code (it is *not at all* easy to develop Lean 4). The more a program is used, the more likely its bugs (if there are any) are to be found. Are there any other bugs in Lean 4? Anyways, subtle bugs can stay hidden for a long time. For more on what is trusted in Lean 4: https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

In this course we assume that Lean 4 has no bugs. So, everything we prove will be assumed true.

Another point is that Lean 4 is not the only proof assistant out there. It is one of the popular ones nowadays.

There are two levels in the world of Lean 4.

(1) user

(2) developer

This course focuses on the first level. As users, we might end up finding ways to improve Lean or Mathlib. One day in the future, we might be able to contribute real Lean code to official libraries.

Suggested reading: Terence Tao's blog now has an entry on Lean:

https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/

Note: Legend says the word "bug" originates from a moth found inside the Harvard Mark II computer in 1947.

https://www.si.edu/object/log-book-computer-bug:nmah_334663

https://daily.jstor.org/the-bug-in-the-computer-bug-story/

https://en.wikipedia.org/wiki/Bug_(engineering)

In fact, the famous inventor Thomas Edison already called faults in machines "bugs" in 1878, long before computers existed.

https://daily.jstor.org/the-bug-in-the-computer-bug-story/

2. Some Terminology

An operating system (OS) is the program on our PC (personal computer) that runs our computer and starts all other programs. Operating systems commonly used include: Windows, Linux, macOS. In this course, we use Lean 4 in Linux, a free OS, so that everyone has to type the same commands in the terminal. Linux has different versions. Popular versions include:

- Debian: https://www.debian.org
- Ubuntu: https://ubuntu.com (Ubuntu is based on Debian)
- Linux Mint: https://linuxmint.com (Linux Mint is based on Ubuntu)

A text-based way to use the operating system is to use a terminal, a program with a window in which we type commands. The program inside the terminal that reads our commands and asks the operating system to execute them is a *shell*. Each operating system comes with its own shell. Linux usually has bash, and Windows has PowerShell. Different shells understand slightly different commands. In this course, a Linux terminal is used with the shell bash.

On a computer that uses Windows 11, there is a way to use Linux: The *Windows Subsystem for Linux* (WSL):

https://learn.microsoft.com/en-us/windows/wsl/about

To install WSL, see:

https://learn.microsoft.com/en-us/windows/wsl/install

With WSL, Windows runs a real Linux system inside a small *virtual machine*, i.e. a separate smaller computer inside the real computer. Windows starts and manages it for us, so it feels as if our operating system is Linux itself. WSL gives Windows users the same terminal as Linux users.

A *repository* (or "repo") is a folder of files whose history of changes is recorded by a program, called a *version control system*, that keeps track of all the changes that happen in a repo. By far the most popular one is git, which our course uses. The changes made in the repo are kept in a hidden folder inside the repo, ".git". (To see this folder in our repo, we need to type "ls -a" in the terminal inside the repo, instead of just "ls".) If a repo is stored on our PC, it's called a *local repo*. Many repos are stored online on the internet (which just means on some other server, somewhere on earth). A repo stored online is called a *remote repo*. GitHub (https://github.com) is a website that stores repos online, publicly or privately. If the repo is public, anyone can download a copy. (There are legal complications about modifying and distributing a repo. These are managed by referring to a file named LICENSE, in the top folder of a repo. In particular, if the license is Apache License 2.0:

https://www.apache.org/licenses/LICENSE-2.0

then anyone may use, change and share its files, as long as they keep the copyright notices, include a copy of the license, and mark each changed file with a note at its beginning.) Our course repo will be a remote Apache-licensed repo on GitHub:

https://github.com/utkuokur/lean-course

Our course repo is "forked" from Dr. Bhavik Mehta's repo:

https://github.com/b-mehta/formalising-mathematics-notes

A *Lean package* is a folder containing Lean files, together with two special files: `lean-toolchain` names the Lean version, and `lakefile.toml` describes the package. Informally, a Lean package is sometimes called a Lean *project*. Our course repository itself is a Lean package.

A *dependency* of a package is another package that it needs. The dependencies are listed in the lakefile, under `[[require]]`, together with the version needed. When we use a package, copies of all its dependencies are downloaded into it, into the folder `.lake/packages/`.

Mathlib is a Lean package containing hundreds of thousands of theorems, maintained by the worldwide Lean community:

https://github.com/leanprover-community/mathlib4

Mathlib is not part of Lean itself. It is a dependency of our course package, so a copy of it will be downloaded into `.lake/packages/mathlib/`.

A *version manager* is a program that keeps several versions of another program installed and automatically uses the right one for each project. Our version manager will be elan:

https://github.com/leanprover/elan

The Lean version of our course is written in the file `lean-toolchain`, in the top folder of our course repository (next to `lakefile.toml`). It contains a single line, `leanprover/lean4:v4.34.1`. When we open our Lean project, elan will read this file and determine the version of Lean to run. Furthermore, elan will install this version of Lean on our machine automatically. In our course, throughout the semester, the Lean version will remain `leanprover/lean4:v4.34.1`. Note that neither Mathlib nor Lean is backward-compatible. This means that if Mathlib version or Lean version is upgraded, previously correct proofs can break, because theorems in Mathlib can get renamed in the newer version and the new Lean version can change the rules for how proofs are written.

The package we currently work in is called the *root* package. Our course repo, the package `lean-course`, is the root package. The root package together with all its dependencies is the *workspace*. Lake downloads all the dependencies into the folder `.lake/packages/` of the root package. Our dependencies include Mathlib, and Mathlib in turn depends on other packages, including Batteries and Aesop, so our workspace contains many dependencies.

A *package manager* is a program that downloads the packages a project needs, in the correct versions. Lean's package manager is Lake. The main Lean package we need for this course is Mathlib.

https://lean-lang.org/doc/reference/latest/Build-Tools-and-Distribution/Lake/

To *build* a Lean project means that Lean checks every file in it (in particular, every proof) and saves the results, so that this work does not have to be repeated later. Once this is done, we have a baseline library of correct theorems, and we start adding more theorems ourselves in new lean files. For each file `XX.lean`, the resulting "built" file is of the form `XX.olean`, and kept in a hidden folder `.lake` which sits at the top folder of our Lean project.

- Mathlib's .olean files are kept in `.lake/packages/mathlib/.lake/build/lib/lean/`
- the .olean files generated by building our own lean files are kept in `.lake/build/lib/lean/`.

When another file imports `XX.lean`, Lean loads `XX.olean` instead of checking `XX.lean` again.

The command `lake exe cache get` downloads Mathlib's `.olean` files, so we do not have to build Mathlib ourselves from scratch. The VS Code command *Lean 4: Project: Fetch Mathlib Build Cache* does the same from within VS Code.

Besides being a package manager, Lake is also a build tool. The command `lake build` builds our Lean project.

A repo usually contains a file named `.gitignore` in its top folder. It lists files and folders that git should *ignore*, i.e. not record and not upload to GitHub. Typically these are large files that can be downloaded or re-created on any computer. Each line of `.gitignore` names one such file or folder.

The folder `.lake` is listed in the `.gitignore` of our course repo, so it is not part of the repo. In particular, our course repo does not contain Mathlib: it only names, in `lakefile.toml`, which Mathlib version it needs, and Lake downloads Mathlib into `.lake/packages/mathlib/` on each computer.

A *module* is a single Lean file: every Lean file in a package is automatically a module, without us having to declare anything. The module's name is obtained from the file's path inside the package by replacing slashes with dots and dropping the ending `.lean`; e.g. the file `FormalisingMathematics2026/Section01logic/Sheet1.lean` forms the module:

`FormalisingMathematics2026.Section01logic.Sheet1`

A module can use the theorems of another module after the command `import`, followed by the module's name, if that module belongs to our package or one of its dependencies. For example, `Solutions/Section02reals/Sheet6.lean` starts with `import FormalisingMathematics2026.Solutions.Section02reals.Sheet5`. Two modules cannot import each other. Not only that, but for any chain of module imports $M_0 \to M_1 \to \ldots \to M_r$, where M_i imports M_{i-1}, it is forbidden for the first module to import the last one. Therefore, module imports cannot form any cycle. A *library* is a collection of modules in a package. Our course package has one library, `FormalisingMathematics2026`, consisting of all modules in the folder `FormalisingMathematics2026`.

A *comment* is text in a file that Lean ignores. It is used only to explain real code. In a Lean file, `--` comments out the rest of a line, and `/-` ... `-/` comments out everything

between the two markers. The shortcut Ctrl+/ adds or removes `--` at the beginning of a line.

3. Proofs and Tactics

A *tactic* is a command that we may use in a Lean proof. When Lean reads a tactic, a program written in Lean runs and auto-generates a part of the proof in question. When we use tactics, our proofs get shorter. The tactics run when Lean reads the file, and they output proofs. The kernel only checks these proofs that are generated by the tactics. It never sees the tactics that created them.

We do not have to trust tactics. From the steps of a tactic proof, Lean builds a proof in its basic language, and the Lean kernel (Section 11) checks that proof. A tactic with a bug can fail to prove a statement, but it cannot make the kernel accept a wrong proof.

One exception is `native_decide`, this is a tactic that adds a new axiom. See Section 13 for more on this tactic.

For a theorem `my_theorem` proved by tactics, `#print my_theorem` in the Lean file shows the proof that the tactics built.

Lean's own tactics are declared in `src/Init/Tactics.lean` and implemented in the folder `src/Lean/Elab/Tactic/` of Lean's repository. On our computer, these files are in the toolchain, in `~/.elan/toolchains/leanprover--lean4---v4.34.1/src/lean/`. They are documented in the *Tactic Reference* of the Lean manual: https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/

Mathlib's tactics are in the folder `Mathlib/Tactic/` of Mathlib's repository. On our computer, they are in `.lake/packages/mathlib/Mathlib/Tactic/`.

The line `import Mathlib.Tactic` at the top of our course files makes all of them available. Lean and Mathlib tactics are listed here: https://leanprover-community.github.io/mathlib4_docs/tactics.html

Inside VS Code, hovering the mouse over a tactic shows its documentation, and the command `#help tactic` followed by a tactic's name, e.g. `#help tactic rw`, prints it in the Infoview.

4. Warnings and Errors

Warnings are different from errors. An error means that Lean could not accept a part of the file, e.g. a proof. The theorems in the rest of the file that do not depend on the erroneous theorem remain unaffected, while a theorem that uses it is treated like a theorem that uses `sorry` (see below).

If there is a warning, usually there is a way to remove the warning. Warnings are usually in yellow color and errors are red in the Infoview.

A *linter* is a program that checks code for matters of style rather than correctness, e.g. lines that are too long or old-fashioned notation. Its warnings do not reject a correct proof.

The name comes from the first such program, *lint*, written by Stephen C. Johnson at Bell Labs in 1978. The name may be connected to the trap of a clothes dryer, as it catches small bits of fluff.

One kind of warning is the *deprecation* warning. It usually says "XX has been deprecated". This means Lean or Mathlib has replaced this name by a new one. The old name still works for a while, but it is a good idea to use the new name to be compatible with future versions of Mathlib.

Another warning is that created by the tactic 'sorry'. This tactic allows us to tell Lean that we are aware that there is a gap in the proof so that when Lean starts checking the proof, it does not raise an error. In particular, we see a yellow colored warning in the Infoview when we use this tactic, instead of a red error.

Putting a "sorry" closes any goal without proving it. Effectively, this postpones the proof for later. If a theorem depends on a "sorry", it is essentially useless, unless there is a way to remove that "sorry".

A theorem that depends on a `sorry` is not proved yet. In this case, `#print axioms` lists the axiom called `sorryAx` for it (q.v. Section 9). In a Lean file, there could be theorems that do not depend on a 'sorry'. Those theorems that do not use sorry remain unaffected, even if they come later in the file. Unlike an error, a `sorry` does not make `lake build` fail. In this sense, having a 'sorry' is less catastrophic than an error.

We need to keep track of which theorems are free of `sorry` before we build further theorems on them. The tactic 'sorry' can be useful, but also requires care.

5. Lean 4 toolchain

Source code is the text of a program as programmers write it, in a programming language that programmers can read. (This is called "higher-level" code). A computer cannot run source code directly. *Compiling* means translating code into a lower-level language, often all the way down to machine code. An *executable* is a program the computer can run directly; to make one from Lean code, Lean's compiler first translates it into C code, a C compiler then turns that into machine code, and finally the pieces are joined (*linked*) into one executable file.

A *toolchain* is the bundle of all programs needed to work with a programming language, all of the same version. The Lean toolchain of our course, `leanprover/lean4:v4.34.1`, contains among others:

- `lean`: checks Lean files; the kernel and the compiler are part of it,
- `lake`: the package manager and build tool,
- the `.olean` files of Lean's own basic libraries

Lean itself is developed in its own repo, maintained by the Lean developers:

https://github.com/leanprover/lean4

Lean's developers publish each version ready to use: the programs (`lean`, `lake`, ...) are already compiled into machine code, and the `.olean` files of Lean's own libraries are already built:

https://github.com/leanprover/lean4/releases/tag/v4.34.1.

elan downloads the compressed file for our operating system (for Linux, `lean-4.34.1-linux.tar.zst`, about 580 MB) and unpacks it into `~/.elan/toolchains/leanprover--lean4---v4.34.1/` (about 3 GB). So we never need Lean's repository or its source code on our computer.

Note: The file `lean-toolchain` in the top folder of our Lean project only *names* the toolchain we need, in one line. The toolchain itself (about 3 GB) is kept in the hidden folder `~/.elan/toolchains/`, in our home folder.

6. VS Code

An integrated development environment (IDE) is a program for writing code that combines a text editor with tools that understand the programming language: it shows errors as we type, explains what names mean, and so on.

Visual Studio Code (VSCode) is a free code editor from Microsoft:

https://code.visualstudio.com.

We add Lean support to VSCode by installing the Lean 4 extension for VSCode. (In VSCode, search for Lean 4 in Extensions panel on the left.) This turns VSCode into an IDE for Lean. After that, when we open a Lean file (a file whose name ends in `.lean`) VSCode recognizes it as a Lean file, and the extension starts Lean in the background (using the version named in `lean-toolchain`). The panel on the right of VS Code that shows the current goal and Lean's messages is called the *Infoview*.

From then on we work interactively, so that Lean checks the file as we type, and shows its messages and the current goal of a proof. This is not the same as building: Lean re-checks the open file only from the place we edited onward, and keeps the results only in memory. This is called "incremental checking" and does not create a new `.olean` file. (The file we are editing gets an `.olean` file only when we build it, for example using the command `lake build`.) The Infoview has a pause button that freezes what the Infoview shows, while Lean keeps checking the file.

7. Setting up Lean and Mathlib

Instructions are here: (Linux/Mac users may use these instructions.)

https://lean-lang.org/install/

Here is a detailed list of instructions for Windows 11 WSL users. About 15 GB of free space would be a safe margin.

(1) **In Windows 11, install WSL**:

(a) Click the Start menu, type "PowerShell", right-click it and choose *Run as administrator*.

(b) Type `wsl --install` and press Enter.

(c) Restart the computer.

(d) Select Ubuntu from start menu. (search for Ubuntu if needed.)

(e) Determine username and password and save them if needed.

Note: RAM (random-access memory) is the computer's fast working memory: programs keep the data they are currently using there, and it is emptied when the computer is switched off (unlike the disk, which keeps our files). With WSL, two operating systems run at the same time, Windows and Linux, and the Linux one runs in a virtual machine that may use a lot of RAM. To give this RAM back to Windows, close VS Code and type `wsl --shutdown` in PowerShell.

(2) **Install VSCode:**

https://code.visualstudio.com.

(3) **Install VSCode WSL Extension**:

(a) In VSCode, open Extensions panel on the left. (If Extension panel is not there, right click on the bar and activate it.)

(b) Search for "WSL".

(c) Find and install the extension called WSL, published by *Microsoft*.

Note: VS Code comes with built-in AI features, e.g. a chat and automatic code suggestions. To disable them, open the settings via:

File → *Preferences* → *Settings* (or just press CTRL + Comma (","))

then, search for "disable AI features" and tick the box, `chat.disableAIFeatures`. This hides the AI features and turns off the Copilot extensions.

(4) **Connect VSCode to WSL**:

(a) Click the button in the bottom-left corner of VSCode.

(b) Choose *Connect to WSL*.

(c) A new VSCode window opens; its bottom-left corner now shows *WSL: Ubuntu*. Use this window from now on.

(5) **Install VSCode Lean Extension**: In VSCode, open Extensions panel on the left. Search for "Lean 4". Find and install the extension called Lean 4, published by *leanprover*.

Note: *leanprover* is the username on github, used by the developers of Lean: https://github.com/leanprover.

(6) **Install Lean**:

(a) Press Ctrl+Shift+P. This opens the *Command Palette*, a search box for all VSCode commands.

(b) Type "Install Dependencies" and choose *Lean 4: Setup: Install Dependencies*. This installs git and curl if they are missing (it may ask for the Linux password from step 1).

(c) In the same way, choose *Lean 4: Setup: Install Lean*. This installs elan, together with the latest Lean version as a default.

The default version does not matter for us: when we open our course project, elan reads `lean-toolchain` and installs the course version (`v4.34.1`) automatically.

(7) **Download the course repository**:

(a) In the Command Palette (Ctrl+Shift+P), choose *Lean 4: Project: Download Project...*

(b) Paste https://github.com/utkuokur/lean-course and choose a folder.

(c) When it has opened, choose *Lean 4: Project: Fetch Mathlib Build Cache*. This downloads Mathlib's `.olean` files. This might take a while.

(8) **Open and check a Lean file**:

(a) On the left bar, click on File Explorer (Ctrl+Shift+E)

(b) Find the file, `FormalisingMathematics2026/Section01logic/Sheet1.lean` and click.

(c) The Infoview opens automatically on the right. If it does not, click the ∀ button in the top-right corner, labeled "Lean 4" and choose *Toggle InfoView* (or just Ctrl+Shift+Enter).

(d) Wait until double blue ticks appear besides `first_theorem` on line 70. We just proved our first theorem in the course!

8. Lakefile

In our course repository, `lakefile.toml` is in the top folder, next to `lean-toolchain`. For Lake, each folder with a lakefile.toml is a Lean package. This includes the top folder.

In the lakefile, `#` comments out a line, instead of `--`.

In our course, lakefile includes the following lines:

- `name = "lean-course"` The name of our root package (or Lean project).
- `defaultTargets = ["FormalisingMathematics2026"]`

  A *target* is something Lake can build, such as a library or a single module. defaultTargets is what 'lake build' in the terminal builds for our course. Our library defined below is the only default target.

- `[leanOptions]`

  Settings defined below apply to every Lean file of the package.

- `pp.unicode.fun = true`

  When this is on, functions are displayed as `fun x ↦ x + 1` instead of `fun x => x + 1`. This is mostly for the Infoview, messages, mouse hovers

- `autoImplicit = false`

  A name that was never introduced before causes an error, instead of silently creating a new variable.

- `relaxedAutoImplicit = false`

  When `relaxedAutoImplicit` is off, only one-letter names like `x`, α, n_1 may become automatic variables, but not long names like `Continous`. This only applies if `autoImplicit` is on. Here it is an extra safety measure.

- `weak.linter.mathlibStandardSet = true`

  This line switches on Mathlib's style linters, which create warnings. The course files switch them off at the top. `weak.` makes the setting conditional: it takes effect in files that import Mathlib, where the option exists, and is ignored in all other files

- `[[require]]` The dependencies. For this course, the only direct dependency is Mathlib:

  - `name = "mathlib"`,
  - `rev = "v4.34.1"`: the version of Mathlib to be used,
  - `git = "https://github.com/leanprover-community/mathlib4.git"`:

    the repo from which Lake downloads Mathlib

- `[[lean_lib]]`

  This line starts the definition of our library. The lines below it describe the library.

- `name = "FormalisingMathematics2026"`

  This is the name of our library

- `globs = ["FormalisingMathematics2026.+"]`:

  `globs` builds every course file on its own, because otherwise name clashes prevent the root module from building. The pattern `FormalisingMathematics2026.+` means all modules inside the folder `FormalisingMathematics2026`, but not the root module `FormalisingMathematics2026.lean` next to it.

A setting line we write inside a lean file (usually at the beginning of the file) overrides the lakefile.

9. Lean 4 Axioms

An *axiom* is a statement that is accepted without proof. Every theorem is ultimately proved from axioms, so the axioms decide what can be proved.

Lean has three standard axioms:

(1) `propext` (*propositional extensionality*):

If two propositions P and Q (terms of type `Prop`) are equivalent then they are equal. In short, $P \leftrightarrow Q$ implies $P = Q$.

(2) `Quot.sound`:

In a quotient $X/\sim$ by a relation $\sim$, if $a \sim b$, then the equivalence classes are equal: $[a] = [b]$.

(3) `Classical.choice`:

If a type is nonempty, then we can choose an element of it.

These three axioms are stated in Lean's basic library `Init`.

`propext` lives in `src/Init/Core.lean`, whereas `Quot.sound` and `Classical.choice` reside in `src/Init/Prelude.lean`. On our computer, these files are found in the toolchain,

`~/.elan/toolchains/leanprover--lean4---v4.34.1/src/lean/Init/`.

A non-standard axiom of Lean we should be aware of is `sorryAx`. This axiom is stated in `src/Init/Prelude.lean` of Lean's repository and it is located at

`~/.elan/toolchains/leanprover--lean4---v4.34.1/src/lean/Init/Prelude.lean`

on our computer. Writing `sorry` uses this axiom. Lean sometimes uses when a part of the file contains an error, by replacing a broken part with a `sorry`, so that it can keep checking the rest of the file. This is why a theorem that uses another broken theorem may depend on `sorryAx`.

10. Linter Warnings

Our lakefile by default switches on Mathlib's style linters, which create warnings, for every file of our package that imports Mathlib. These are intended as rules that Mathlib contributors must follow. For example, if a line has more than 100 characters, it is considered too long and we get a warning in the Infoview.

We manually turn off linter warnings of mathlib and Lean at the start of the course, by adding the line

```
set_option linter.all false  -- comment out to see Mathlib's style warnings
```

This works because a setting written inside a file overrides the lakefile. To see the linter warnings in a course file, we can *comment out* the line by putting `--` at its beginning.

New files that we create ourselves do not contain the line `set_option linter.all false`, so we will start seeing more linter warnings. This is good for us, because as we get more comfortable with Lean, we are expected to write our proofs in Mathlib's style.

11. Lean 4 kernel

The kernel is written in C++. Its source code is in the folder `src/kernel` of Lean's repository. For our Lean version, its code can be found here:

https://github.com/leanprover/lean4/tree/v4.34.1/src/kernel

(the main file is `type_checker.cpp`). The source code is not included when elan installs Lean: on our computer the kernel exists only in compiled form, inside the file `~/.elan/toolchains/leanprover--lean4---v4.34.1/lib/lean/libleanshared.so`.

12. Useful Git Commands

Type these commands in the terminal, inside the folder of the repo. Parts in angle brackets, like `<url>`, are placeholders: we replace them, including the brackets, by our own text.

- `git clone <url>`

  download a copy of the repo at `<url>` into a new folder.

  The new folder is named after the last part of `<url>`

  `git clone <url> <folder>` chooses the folder's name

- `git pull`

  download the newest changes of the remote repo (e.g. new exercise sheets) and add them to our files

- `git status`

  list the files we changed since the last commit

- `git diff`

  show these changes line by line

- `git restore <file>`

  throw away our changes in `<file>` (this cannot be undone)

- `git add <file>`

  choose `<file>` for the next commit

- `git commit -m "<message>"`

  record the chosen changes in the history, with a short message

- `git log --oneline`

  list the commits, one line each, newest first

- `git branch`

  list our branches on this computer

- `git branch -a`

  list our branches and the branches of the remote repo

- `git remote -v`

  show the remote repos that our local repo knows, with their addresses (the course repo is called `origin`)

- `git remote add <name> <url>`

  add our own repo on GitHub at `<url>` as a second remote repo, with a name of our choice, `<name>`

- `git push -u <name> main`

  upload our commits to the remote repo `<name>` for the first time; `-u` makes it the default for later pushes and pulls

- `git push`

  upload our new commits to the default remote repo

- `git pull origin main`

  download the newest changes of the course repo (after `git push -u <name> main`, a plain `git pull` downloads from `<name>` instead)

13. Advanced Topics: Compiler, Interpreter and Runtime

Lean is not only a proof assistant, but also a programming language. A definition such as

```
def f (n : Nat) : Nat := n * n + 1
```

can be turned into a small program, and we can run it. For example, if we add the command line `#eval f 10` in the Lean file containing the definition above, then the Infoview shows `101`.

A *processor* is a computer chip that carries out instructions, such as adding and comparing numbers or moving data in memory. The processor only understands "machine code" and different kinds of processors have different machine code. In fact, Lean is published separately for each kind of processor. The central processing unit (CPU) of a computer is the main processor of a computer. It runs the operating system and almost all programs. A *GPU* (graphics processing unit) is a different kind of a processor, with thousands of small cores that carry out the same simple calculation on many numbers at once. It was initially designed for drawing images, but it can also be used to run artificial intelligence (AI) programs. Lean does not need a GPU. It runs entirely on the CPU.

So source code must be translated into machine code, and there are two ways to do this. Think of a speech in a foreign language:

- A *translator* translates the whole text in advance. Afterwards, anyone can read the translation quickly, without the translator. This is a *compiler*: it translates a whole program into machine code before the program runs. The result, a file that the computer can run directly, is an *executable*. Lean has its compiler, see below.
- An *interpreter* at a conference translates sentence by sentence, while the speaker talks. Nothing is written down, and it is slower, but it starts immediately. This is an *interpreter* in computing: a program that reads code and carries it out step by step, right away. Lean has its interpreter, see below.

In both cases, the running program needs some help, for example to store data in memory, or to compute with numbers that are too large for the processor, such as $f(10^{10}) = 100000000000000000001$. This helper code is called the *runtime*. The runtime helps both the interpreter and every executable while they run. Figure 1 gives an overview.

13.1. Lean 4 compiler.

The Lean compiler is written in Lean itself. Its source code is in the folder `src/Lean/Compiler` of Lean's repository:

https://github.com/leanprover/lean4/tree/v4.34.1/src/Lean/Compiler.

Unlike the kernel, this source code is included when elan installs Lean. That means we can read the Lean compiler's source code on our own computer:

`~/.elan/toolchains/leanprover--lean4---v4.34.1/src/lean/Lean/Compiler/`.

What Lean compiler does is, it first translates our definitions into a simpler *intermediate form*, which does not depend on the processor yet. From there, there are two ways on:

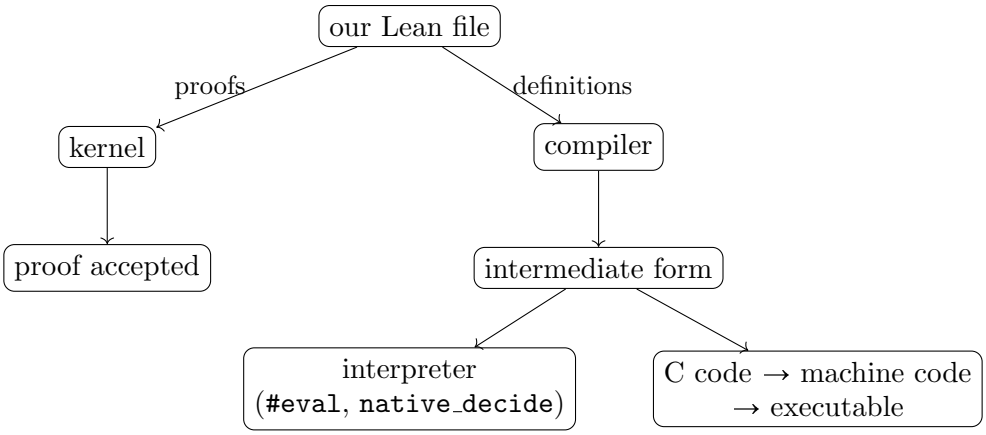


FIGURE 1. What happens to the proofs and the definitions in our Lean file.

(1) To make an executable, the compiler translates the intermediate form into C code; a C compiler (`clang`, which comes with the toolchain) translates the C code into machine code; finally, the pieces are joined (*linked*) with the runtime into one executable file. For example, `lake exe cache get` runs the executable `.lake/packages/mathlib/.lake/build/bin/cache`, which was made in this way from Mathlib's Lean code.

(2) `#eval` and `native_decide` do not make an executable: they hand the intermediate form to the interpreter.

Notes: Proofs are never compiled: the compiler only translates definitions that compute something, and leaves proofs to the kernel. In computer science language, compiling a Lean file means turning it into a computer program that a computer can run. But people still sometimes use the terminology "Compiling a Lean 4 proof". However, we do not mean that the proof was sent to the Lean compiler, we just mean that Lean accepted the proof.

13.2. **The Lean interpreter.** Lean's interpreter is written in C++ and located at `src/library/ir_interpreter.cpp` in Lean's repository. The compiler translates a definition into an intermediate form, and the interpreter runs that intermediate form step by step, inside the running `lean` program (the same program that checks our Lean file), without turning it into an executable.

This is why `#eval f 10` shows `101` in Infoview immediately, without creating any file.

Where possible, the interpreter calls machine code that already exists. Most of Lean's own library comes already compiled with the toolchain, so for example the multiplication of natural numbers runs as machine code.

13.3. **The runtime.** The runtime is written in C and C++ and located in the folder `src/runtime` in Lean's repository. It comes with the toolchain as machine code. It manages the memory of a running program (it frees data that is no longer needed), computes with arbitrarily large numbers, and talks to the operating system, e.g. to print text or to read files.

13.4. **Trust.** The compiler, the interpreter and the runtime are large and complicated, because they try to make programs run fast, so they are harder to check for bugs than the small kernel. A bug in them could make a computation give a wrong result. For proofs this does not matter, because proofs are checked by the kernel alone. One exception is the tactic `native_decide`. It proves a statement such as $12345 \cdot 6789 = 83810205$ by computing: the compiler turns the check (multiply, then compare) into a program, the interpreter runs it, and Lean trusts the answer, instead of letting the kernel check the computation. This is faster, but then the proof also trusts the compiler, the interpreter and the runtime. Lean records this with an automatically created axiom: for a theorem `by_native` proved in this way, `#print axioms` lists the axiom `by_native._native.native_decide.ax_1_1` (Section 9). For this reason, Mathlib does not allow `native_decide`, while some other projects accept it.